Ingenieria Informatica

Procesadores de Lenguaje 25/26
Grupo 81

Top-Down

«Calculadora»

121:

Denis Loren Moldovan —
100522240@alumnos.uc3m.es
Jorge Adrian Saghin Dudulea —
100522257@alumnos.uc3m.es

Profesor
Maria Paz

Tabla de contenidos

1. **Cuestiones** 2
1.1. Funciones de lexer 2
1.2. Gramática 2
1.2.1. Gramática inicial 2
1.2.2. Equivalente de jflap 2
1.2.3. Condiciones LL(1) 3
1.2.4. Distinción entre niveles léxico y sintáctico 3
1.3. Diagrama sintáctico 4
1.4. Conjunto de pruebas 4
1.4.1. Variables y números simples 4
1.4.2. Operaciones aritméticas simples 5
1.4.3. Expresiones compuestas 6
1.4.4. Asignación simple 7
1.4.5. Asignación encadenada 7
1.4.6. Ternario 8
1.4.7. Asignación ternaria (ContAsignacion $\rightarrow EE$) 8
1.4.8. Combinaciones avanzadas 9
1.4.9. Edge cases 10

1. Cuestiones

1.1. Funciones de lexer

El lexer va leyendo caracteres, hasta la terminación del programa mediante señales (`Ctrl+C`), o hasta EOF (Input mediante ficheros usando el operador de entrada <) facilitando las pruebas automatizadas. Este se va saltando las tabulaciones, espacios en blanco, y saltos de línea (Cuando este último ocurre, aumenta el contador de líneas).

Cuand se lee cualquier elemento, se actualiza el token anterior ($LL(k)/k = 1$).

- Mientras sea un ńumero, simplemente devuelve el token de dígito.
- Cuando sea un caracter, este lee el siguiente, lo almacena junto con el token como información sumplimentaria, y devuelve el token de variable.
- Si es un operador, ocurre igual que con los dígitos, y lo devuelve directamente.
- Finalmente, si no ramificamos de ninguna manera, el token se devuelve como literal.

1.2. Gramática

```
Axioma              ::= Expresion
Expresion           ::= (OperadorExpresion) | Numero | Variable
OperadorExpresion   ::= Op Expresion Expresion | Asignacion | Ternario
Op                  ::= + | - | * | /
Asignacion          ::= = Variable Expresion ContAsignacion
ContAsignacion      ::= Expresion Expresion | #
Ternario            ::= ? Expresion Expresion Expresion
Numero              ::= 0 | 1
Variable            ::= a | b
```

1.2.1. Gramática inicial

A diferencia de la primera gramática implementada durante la primera clase, hemos añadido las reglas de las asignaciones tanto normales como ternarias. Por otro lado, en el inicial, implementamos las variables sacandolas directamente del buffer, sin tomar en cuenta de que estos vienen dados por el lexer en forma de token.

```
Axioma              ::= Expresion
Expresion           ::= (Operador Expresion) | Numero | Variable
Operador Expresion  ::= + Expresion Expresion
Numero              ::= 0 | 1
Variable            ::= a Contvariable
Contvariable        ::= a | 0 | #
```

1.2.2. Equivalente de jflap

```
A -> E
E -> (O)
E -> N
E -> V
O -> +EE
N -> 0
N -> 1
```

```
V -> a
V -> b
```

1.2.3. Condiciones LL(1)

La gramática satisface las condiciones LL(1) porque en cada producción con alternativas, los conjuntos FIRST de cada alternativa son disjuntos, lo que permite seleccionar la producción correcta con un único token de lookahead.

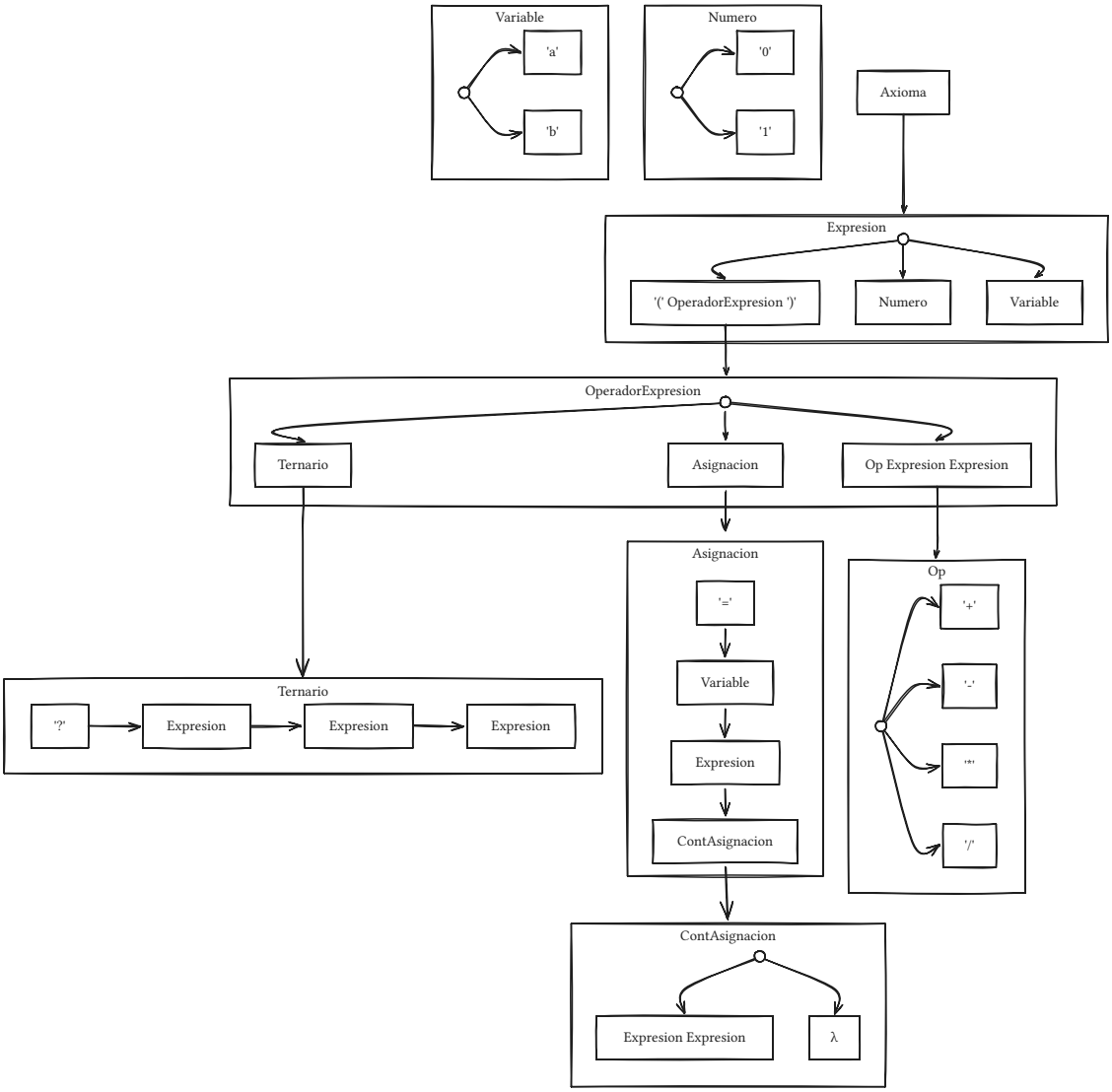
	FIRST	FOLLOW
A	{ 0, 1, a, b, (}	{ $ }
C	{ λ, 0, 1, a, b, (}	{) }
E	{ 0, 1, a, b, (}	{ 0, 1, a, b, $, (,) }
N	{ 0, 1 }	{ 0, 1, a, b, $, (,) }
O	{ *, +, -, =, /, ? }	{) }
T	{ ? }	{) }
V	{ a, b }	{ 0, 1, a, b, $, (,) }
W	{ *, +, -, / }	{ 0, 1, a, b, (}
X	{ = }	{) }

1.2.4. Distinción entre niveles léxico y sintáctico

El diseño del traductor separa explícitamente dos niveles de análisis. El nivel léxico, implementado en `rd_lex()`, opera sobre caracteres individuales y los agrupa en unidades significativas llamadas tokens: números (`T_NUMBER`), variables (`T_VARIABLE`), operadores aritméticos (`T_OPERATOR`) y literales sueltos como `(`, `)`, `=`, `?`. Este nivel no conoce ninguna regla gramatical; únicamente aplica patrones de reconocimiento sobre el flujo de caracteres.

El nivel sintáctico, implementado en las funciones `Parse*`, opera exclusivamente sobre la secuencia de tokens producida por el lexer. Cada función de parse comprueba el token actual (`tokens.token`) para decidir qué producción aplicar, sin acceder nunca al flujo de caracteres directamente. Esta separación tiene varias ventajas: el lexer puede modificarse de forma independiente (por ejemplo, para ampliar el alfabeto de variables o el rango de números) sin alterar la gramática, y el parser puede razonar sobre la estructura de la expresión sin preocuparse por espacios en blanco, tabulaciones o saltos de línea, que el lexer ya ha descartado.

1.3. Diagrama sintáctico



1.4. Conjunto de pruebas

Las pruebas se organizan en categorías progresivas, desde tokens simples hasta expresiones complejas con anidamiento profundo y combinaciones de todas las construcciones de la gramática.

1.4.1. Variables y números simples

Verificación de que el lexer reconoce correctamente todos los tokens atómicos: variables de una y dos letras (con y sin dígito), y números de uno o varios dígitos.

Entrada	Salida
A	A
B	B
a	a
b	b

z	z
Z	Z
0	0
1	1
42	42
321	321
1234	1234
9999	9999
a1	a1
b2	b2
z7	z7
A2	A2
B3	B3
AB	AB
CD	CD
az	az
bz	bz
A1	A1
Z9	Z9

1.4.2. Operaciones aritméticas simples

Operaciones binarias planas con operandos literales: números, variables mayúsculas, variables minúsculas, y combinaciones mixtas (variable + número, número + variable, variables de dos caracteres).

Entrada	Salida
(+ 1 2)	(1 + 2)
(- 5 3)	(5 - 3)
(* 2 4)	(2 * 4)
(/ 8 2)	(8 / 2)
(+ 0 1)	(0 + 1)
(- 1 0)	(1 - 0)
(* 1 1)	(1 * 1)
(/ 2 1)	(2 / 1)
(+ A B)	(A + B)
(- A B)	(A - B)
(* A B)	(A * B)

`(/ A B)`	`(A / B)`
`(+ a b)`	`(a + b)`
`(* a b)`	`(a * b)`
`(- a b)`	`(a - b)`
`(/ a b)`	`(a / b)`
`(+ A 1)`	`(A + 1)`
`(- B 2)`	`(B - 2)`
`(* A 42)`	`(A * 42)`
`(/ B 10)`	`(B / 10)`
`(+ 1 A)`	`(1 + A)`
`(- 2 B)`	`(2 - B)`
`(+ A2 B3)`	`(A2 + B3)`
`(* AB CD)`	`(AB * CD)`
`(+ a1 b2)`	`(a1 + b2)`
`(- az bz)`	`(az - bz)`
`(* A2 B)`	`(A2 * B)`
`(/ AB 2)`	`(AB / 2)`

1.4.3. Expresiones compuestas

Expresiones con subexpresiones anidadas, comprobando que la precedencia y los paréntesis se preservan correctamente en la salida infija.

Entrada	Salida
`(+ 1 (* 2 3))`	`(1 + (2 * 3))`
`(* (+ 1 2) 3)`	`((1 + 2) * 3)`
`(- (* 2 3) 1)`	`((2 * 3) - 1)`
`(/ (+ 4 2) 3)`	`((4 + 2) / 3)`
`(+ (* 2 3) (* 4 5))`	`((2 * 3) + (4 * 5))`
`(- (+ 1 2) (+ 3 4))`	`((1 + 2) - (3 + 4))`
`(* (- 5 3) (+ 1 2))`	`((5 - 3) * (1 + 2))`
`(/ (* 4 3) (+ 2 1))`	`((4 * 3) / (2 + 1))`
`(+ (+ 1 2) 3)`	`((1 + 2) + 3)`
`(+ 1 (+ 2 3))`	`(1 + (2 + 3))`
`(* (* 2 3) 4)`	`((2 * 3) * 4)`
`(* 2 (* 3 4))`	`(2 * (3 * 4))`
`(+ (+ (+ 1 2) 3) 4)`	`(((1 + 2) + 3) + 4)`
`(* (+ 1 (* 2 3)) (- 4 (/ 6 2)))`	`((1 + (2 * 3)) * (4 - (6 / 2)))`

`(+ A (* B 4))`	`(A + (B * 4))`
`(* (+ A 1) B)`	`((A + 1) * B)`
`(- A (* B C))`	`(A - (B * C))`
`(+ (* A B) (* C D))`	`((A * B) + (C * D))`

1.4.4. Asignación simple

La regla ContAsignacion $\rightarrow \lambda$ produce una asignación directa `variable = expresion`.

Entrada	Salida
`(= a 1)`	`a = 1`
`(= b 2)`	`b = 2`
`(= z 0)`	`z = 0`
`(= A 1)`	`A = 1`
`(= B 42)`	`B = 42`
`(= a b)`	`a = b`
`(= A B)`	`A = B`
`(= a (+ 1 2))`	`a = (1 + 2)`
`(= b (* 3 4))`	`b = (3 * 4)`
`(= A (- 5 3))`	`A = (5 - 3)`
`(= B (/ 8 2))`	`B = (8 / 2)`
`(= A (+ A 1))`	`A = (A + 1)`
`(= B (* B 2))`	`B = (B * 2)`
`(= a (+ a b))`	`a = (a + b)`
`(= b (* a b))`	`b = (a * b)`
`(= A (* 1234 B))`	`A = (1234 * B)`
`(= AB (* 2 B))`	`AB = (2 * B)`
`(= a1 (+ a 1))`	`a1 = (a + 1)`
`(= b2 (* b 2))`	`b2 = (b * 2)`
`(= a (+ (* 2 3) (* 4 5)))`	`a = ((2 * 3) + (4 * 5))`
`(= b (- (* A B) (+ C 1)))`	`b = ((A * B) - (C + 1))`

1.4.5. Asignación encadenada

Cuando el valor asignado es a su vez una asignación, se produce un encadenamiento. La expresión `(= a (= b v))` asigna primero `b = v` y luego asigna ese mismo valor a `a`.

Entrada	Salida
`(= a (= b 1))`	`a = b = 1`
`(= a (= b 2))`	`a = b = 2`

`(= A (= B 42))`	`A = B = 42`
`(= a (= b (= c 1)))`	`a = b = c = 1`
`(= a (= b (= c (= d 1))))`	`a = b = c = d = 1`
`(= a (= b (= c3 (+ 2 3))))`	`a = b = c3 = (2 + 3)`
`(= az (= b (= c3 (+ 2 3))))`	`az = b = c3 = (2 + 3)`
`(= A (= B (= C (+ 1 2))))`	`A = B = C = (1 + 2)`
`(= a (+ (= b 2) (= c 3)))`	`a = (b = 2) + (c = 3)`
`(= a (* (= b 2) (= c 3)))`	`a = (b = 2) * (c = 3)`
`(= A (+ (= B 1) (= C 2)))`	`A = (B = 1) + (C = 2)`

1.4.6. Ternario

La forma `(? cond then else)` se traduce a `cond ? then : else`. Se cubren condiciones literales, compuestas y con asignaciones embebidas.

Entrada	**Salida**
`(? a b c)`	`a ? b : c`
`(? 1 2 3)`	`1 ? 2 : 3`
`(? A B C)`	`A ? B : C`
`(? a 1 2)`	`a ? 1 : 2`
`(? 0 a b)`	`0 ? a : b`
`(? A 1 B)`	`A ? 1 : B`
`(? (+ a 1) b c)`	`(a + 1) ? b : c`
`(? (- a 1) b c)`	`(a - 1) ? b : c`
`(? (* a 2) b c)`	`(a * 2) ? b : c`
`(? a (+ b 1) c)`	`a ? (b + 1) : c`
`(? a (- b 1) c)`	`a ? (b - 1) : c`
`(? a b (+ c 1))`	`a ? b : (c + 1)`
`(? a (+ b 1) (- c 1))`	`a ? (b + 1) : (c - 1)`
`(? (+ a 1) (+ b 1) (+ c 1))`	`(a + 1) ? (b + 1) : (c + 1)`
`(? (* A B) (+ C 1) (- D 1))`	`(A * B) ? (C + 1) : (D - 1)`
`(? (= a 1) b c)`	`a = 1 ? b : c`
`(? a (= b 1) c)`	`a ? b = 1 : c`
`(? a b (= c 1))`	`a ? b : c = 1`

1.4.7. Asignación ternaria (ContAsignacion $\rightarrow EE$)

La regla ContAsignacion $\rightarrow EE$ produce la forma `variable = (cond ? then : else)`, diferenciandose de la asignación simple por el lookahead después de la primera expresión.

Entrada	Salida
`(= a 1 2 3)`	`a = (1 ? 2 : 3)`
`(= b 0 1 2)`	`b = (0 ? 1 : 2)`
`(= A 1 2 3)`	`A = (1 ? 2 : 3)`
`(= a (+ 1 2) 3 4)`	`a = ((1 + 2) ? 3 : 4)`
`(= a 1 (+ 2 3) 4)`	`a = (1 ? (2 + 3) : 4)`
`(= a 1 2 (+ 3 4))`	`a = (1 ? 2 : (3 + 4))`
`(= a (+ b 1) (+ a 1) (? a b c))`	`a = ((b + 1) ? (a + 1) : (a ? b : c))`
`(= x 1 2 3)`	`x = (1 ? 2 : 3)`
`(= a (+ 1 2) (* 3 4) (- 5 6))`	`a = ((1 + 2) ? (3 * 4) : (5 - 6))`
`(= A (* B 2) (+ A 1) (- A 1))`	`A = ((B * 2) ? (A + 1) : (A - 1))`
`(= a (= b 1) 2 3)`	`a = (b = 1 ? 2 : 3)`
`(= a 1 (= b 2) 3)`	`a = (1 ? b = 2 : 3)`
`(= a 1 2 (= b 3))`	`a = (1 ? 2 : b = 3)`

1.4.8. Combinaciones avanzadas

Expresiones que combinan asignaciones, ternarios y operaciones aritméticas en múltiples niveles de anidamiento.

Entrada	Salida
`(+ (= a 1) (= b 2))`	`(a = 1) + (b = 2)`
`(* (= A 1) (= B 2))`	`(A = 1) * (B = 2)`
`(+ (= a (+ 1 2)) b)`	`(a = (1 + 2)) + b`
`(* (= A (* B 2)) C)`	`(A = (B * 2)) * C`
`(+ (= a 1) (* b 2))`	`(a = 1) + (b * 2)`
`(- (* (= a 2) b) (+ c 1))`	`((a = 2) * b) - (c + 1)`
`(* (= x (+ 1 2)) (= y (* 3 4)))`	`(x = (1 + 2)) * (y = (3 * 4))`
`(+ (* (= a 1) b) (* (= c 2) d))`	`((a = 1) * b) + ((c = 2) * d)`
`(= a (+ (= b (* 2 3)) (= c (- 5 1))))`	`a = (b = (2 * 3)) + (c = (5 - 1))`
`(? (+ a b) (* a b) (- a b))`	`(a + b) ? (a * b) : (a - b)`
`(? (= a 1) (+ a 2) (- a 2))`	`a = 1 ? (a + 2) : (a - 2)`
`(= x (+ a 1) (* b 2) (? a b c))`	`x = ((a + 1) ? (b * 2) : (a ? b : c))`
`(= A (- B 1) (* A 2) (? A B C))`	`A = ((B - 1) ? (A * 2) : (A ? B : C))`
`(+ (? a 1 2) (? b 3 4))`	`(a ? 1 : 2) + (b ? 3 : 4)`

`(* (? a b c) (? d e f))`	`(a ? b : c) * (d ? e : f)`
`(= a (? b c d) (? e f g) (? h i j))`	`a = ((b ? c : d) ? (e ? f : g) : (h ? i : j))`
`(+ (= a (? b 1 2)) (= c (? d 3 4)))`	`(a = (b ? 1 : 2)) + (c = (d ? 3 : 4))`

1.4.9. Edge cases

Pruebas de estrés con anidamiento profundo y combinación de todas las construcciones simultáneamente.

Entrada	Salida
`(* (+ 1 (* 2 3)) (- 4 (/ 6 2)))`	`((1 + (2 * 3)) * (4 - (6 / 2)))`
`(= a (= b (= c (= d (= e 1)))))`	`a = b = c = d = e = 1`
`(+ (+ (+ (+ 1 2) 3) 4) 5)`	`((((1 + 2) + 3) + 4) + 5)`
`(* (= a (+ 1 2)) (= b (* (= c 3) 4)))`	`(a = (1 + 2)) * (b = ((c = 3) * 4))`
`(? (= a 1) (= b (+ a 1)) (= c (* a 2)))`	`a = 1 ? b = (a + 1) : c = (a * 2)`
`(= z (+ (* a b) (* c d)) (- (* a b) (* c d)) (? a b c))`	`z = ((a * b) + (c * d)) ? ((a * b) - (c * d)) : (a ? b : c)`